

NEXTONES — Mode d'emploi rapide

Paramètres, règles d'allocation, rebalances, reset et ajout de ticker — version post-Jalon 3 (25 mai 2026)

1. Paramètres réglables et où les modifier

A. Constantes dans execution_engine.py

Édition directe de C:\Users\RichardGUELIN\Prod\ThesiumDesk\execution_engine.py puis redémarrage uvicorn.

Paramètre	Ligne	Rôle (valeur actuelle)
BUILD_INVESTED_THRESHOLD	~110	% NAV investi sous lequel régime=BUILD (20.0)
BUILD_MIN_BUY_FLOOR_PCT	~116	Plancher de poids cible en BUILD (0.35%)
BUILD_MIN_BUY_CONV_THRES HOLD	~116	Conviction mini pour appliquer le plancher (6.0)
MIN_TRADE_WEIGHT_PCT	~147	Seuil sous lequel reconciler DROP la proposition (0.3%)
BUILD_TARGET_OVERRIDE_CO NV	~157	Conviction qui déclenche override cible BUILD (6.0)
BUILD_MIN_SIZE_OVERRIDE_C ONV	~157	Conviction qui force la taille mini en BUILD (6.0)
MAX_OVERSHOOT_TARGET_M ULT	~168-169	Multiplicateur overshoot : BUILD 1.50 / MAINTAIN 1.20 / REBAL 1.50
[BUILD_QTY1_V1] override	~444-453	Equities BUILD : qty=1 si calc>=1, skip sinon
CRYPTO_SET	(recherche)	BTC/ETH/LINK/SOL/ADA/DOT/MATIC/AVAX — fractions 6 décimales

B. Paramètres du construction agent (BDD)

Table target_construction_config, colonne params_json, WHERE id=1. Relus à chaque cycle — pas de redémarrage nécessaire.

```
UPDATE target_construction_config
SET params_json = json_set(params_json,
'$w_conviction', 0.40,
'$w_realized', 0.15,
'$w_macro', 0.25,
'$w_diversif', 0.20,
'$lambda_vol', 0.50,
'$beta_temp', 1.00,
'$budget_build', 0.95,
'$budget_maintain', 0.80,
'$budget_rebalance', 0.90,
'$min_score_inclusion', 0.30,
'$enable_realized', 1
)
WHERE id=1;
```

2. Règles d'allocation et de proposition par phase

Détermination du régime (en début de cycle)

- BUILD : $\text{invested_pct} < \text{BUILD_INVESTED_THRESHOLD}$ (20%)
- MAINTAIN : $20\% \leq \text{invested_pct} \leq 90\%$
- REBALANCE : $\text{invested_pct} > 90\%$ ou drift important sur positions existantes

Score composite par ticker

$$\text{score} = w_{\text{conviction}} \cdot C + w_{\text{realized}} \cdot R + w_{\text{macro}} \cdot M + w_{\text{diversif}} \cdot D - \lambda_{\text{vol}} \cdot V$$

Seuil d'inclusion : $\text{score} \geq \text{min_score_inclusion}$ (0.30). Détail des composantes (R_norm, C_norm, M_norm, D_norm, V_norm) sauvegardé dans `portfolio_targets_history.components_json`.

Règles spécifiques par régime

Régime	Budget brut	Règles principales
BUILD (sous-investi)	95% NAV	Plancher 0.35% si conviction ≥ 6 · Override cible et taille mini si $\text{conv} \geq 6$ · Overshoot $\times 1.50$ · Equities : $\text{qty}=1$ forcée si $\text{calc} \geq 1$, skip sinon [BUILD_QTY1_V1] · Crypto : qty fractionnaire 6 décimales [RDC_CRYPTOV1]
MAINTAIN (croisière)	80% NAV	Overshoot resserré $\times 1.20$ · Pas d'override agressif · Trades sous MIN_TRADE_WEIGHT_PCT (0.3%) ignorés
REBALANCE	90% NAV	Overshoot $\times 1.50$ · Max sell BUILD : 25% par cycle (limite de rotation) · Tickers en surpoids ramenés vers la cible sans dépasser le cap

3. Rebalances et prises de profit

Mécanisme	Lieu	Comportement
Seuil d'action mini	MIN_TRADE_WEIGHT_PCT (0.3%)	En dessous, le reconciler DROP la proposition
Cap par ticker	MAX_OVERSHOOT_TARGET_MULT	Empêche de dépasser cible × multiplicateur du régime
Cap/floor appliqué	portfolio_targets_history.cap_floor_applied	Trace booléenne dans le snapshot
Limite de vente cycle	Max sell BUILD = 25%/cycle	Évite la liquidation brutale
Vente automatique	Reconciler	Déclenchée quand poids actuel > cible × overshoot — pas de stop-loss prix
Trace décisionnelle	Table cycle_reconciliation_log	action, reason, delta_signal_pct, delta_target_pct

Note : il n'y a pas de prise de profit explicite par seuil PnL — la rotation se fait par dérive de score et dépassement d'overshoot.

4. Reset du portefeuille à 100% cash

Procédure de production (1M EUR de cash, purge complète des 10 tables trading) :

```
cd C:\Users\RichardGUELIN\Prod\ThesiumDesk
powershell -ExecutionPolicy Bypass -File .\reset_portfolio_full.ps1
```

Ce que fait le script

- Backup automatique de thesium.db
- Purge : orders, theses, portfolio_positions, portfolio_targets, portfolio_targets_history, cycle_reconciliation_log, construction_snapshots, portfolio_history (sauf ligne reset), agent_decisions, validation_queue
- Insert portfolio_state : cash=1 000 000, total_value=1 000 000, PnL=0
- Insert portfolio_history avec date='today' [RESET_DATE_FIX_V1]

Puis relancer l'API et un cycle

```
py -3.13 -m uvicorn api_server_with_static:app --host 0.0.0.0 --port 8000
```

Cliquer ensuite Run Decision Cycle dans l'UI (endpoint POST /api/orders/execute-cycle protégé par auth).

5. Ajout d'un ticker en portefeuille

Étape 1 — Inscription dans l'univers

```
-- Vérifier qu'il n'existe pas déjà
SELECT id, ticker FROM universe WHERE ticker='XXX';

-- Insertion (asset_class: equity, crypto, etf, bond...)
INSERT INTO universe (ticker, name, asset_class, sector, active)
VALUES ('XXX', 'Nom complet', 'equity', 'Technology', 1);

-- Si crypto, ajouter à CRYPTO_SET dans execution_engine.py
```

Étape 2 — Table instruments (cohérence FK)

```
INSERT OR IGNORE INTO instruments (id, ticker, name, asset_class)
SELECT id, ticker, name, asset_class FROM universe WHERE ticker='XXX';
```

Étape 3 — Charger l'historique 90 jours

Indispensable pour le calcul de R_norm (Sharpe 90j) et de V_norm (vol).

```
powershell -ExecutionPolicy Bypass -File .\nextones-enrich-prices-90d.ps1 -Ticker XXX
```

Vérification :

```
SELECT ticker, COUNT(*) AS nbj, MIN(date), MAX(date)
FROM prices WHERE ticker='XXX';
-- Doit retourner nbj >= 84
```

Étape 4 — Générer une thèse initiale

Soit via l'UI (Add Thesis), soit en SQL :

```
INSERT INTO theses (instrument_id, agent_type, thesis_text, conviction_score,
horizon, proposed_action, status)
VALUES ((SELECT id FROM universe WHERE ticker='XXX'),
'manual', 'Texte de thèse', 6.5, 'medium', 'BUY', 'active');
```

Étape 5 — Forcer le snapshot et observer

```
powershell -ExecutionPolicy Bypass -File .\nextones-regen-snapshot-with-R.ps1
```

Puis Run Decision Cycle dans l'UI. Vérifier dans portfolio_targets_history que le ticker apparaît avec score ≥ 0.30 et included=1.

Annexe — Vérifications rapides en console

```
# Dernier cycle et ordres BTC/crypto
py -3.13 .\verif_btc_orders_table.py

# Pourquoi un ticker est absent
py -3.13 .\diag_link_missing_v2.py LINK

# Inspecter les R_norm du dernier snapshot
py -3.13 .\diag_realized_v2.py
```

Marqueurs à surveiller dans les logs uunicorn

```
[RDC_CRYPT0_V1] · [BUILD_QTY1_V1] · [score_R] · [CAPPED] · [target_gap] ·
[RESET_DATE_FIX_V1]
```